INDEPENDENT SECURITY RESEARCH NOTE

Looking Beyond the Patch: PHP cURL Push-Callback Replacement

A source-level lifecycle hypothesis backed by a control-versus-failure crash observation.

Observed runtime	PHP 8.4.14 NTS	**HTTP stack**	libcurl 8.14.1 / HTTP/2
Reported platform	Alpine Linux 3.21.9 under proot on Android	**Prepared**	27 October 2026

Executive summary

A local PHP cURL test exhibited a segmentation fault after a valid `CURLMOPT_PUSHFUNCTION` callback was registered, the caller dropped its external reference to the receiver, a replacement with `null` raised `TypeError`, and the same multi handle was driven against a local HTTP/2 Server Push fixture. The matching valid-callback control invoked the callback and exited normally. The failing transcript did not print a userland callback invocation, so the precise faulting instruction remains unobserved.

Current conclusion: a crash candidate has been observed in one configuration. This report does not establish use-after-free, memory corruption, code execution, remote exploitability, an affected-version range, a CVE, or critical severity.

Why investigate this sequence?

The interesting feature is the ordering of operations. Current php-src source destroys the existing Zend callable cache before validating the proposed replacement; the invalid-input path returns before registering a new callback. The native libcurl trampoline may therefore remain configured while PHP-side callback state is no longer initialized. The source order is consistent with the crash, but a memory-safety trace is still needed.

Scope

This is a separate cURL-extension investigation. PHP's October 2026 OOM and stale-worker-state disclosures motivate the lifecycle-audit method, but they are not presented as the same bug or as evidence of comparable severity.

Prepared as a reproducibility and review artifact. The raw console transcripts were provided during the research session and were not independently re-run during archive assembly.

1. Observed behavior and reproduction

The package contains a shared PHP probe with two modes. Both modes use a local cleartext HTTP/2 server configured to push `/push.txt` with the parent resource `/index.html`. The server binds to loopback only.

Mode	Observed output	Interpretation
Control	Callback invoked; multi status reported as no error; process exit 0.	The recorded fixture reached the userland server-push callback.
Invalid replacement	TypeError; receiver destructor; add status 0; segmentation fault; exit 139. Callback invocation was not logged.	Crash correlated with a failed callback replacement sequence; precise fault site is unknown.
Recovery attempt	Connection refused; HTTP status 0; no replacement callback invoked.	Excluded as invalid evidence because the server was not running.

Run instructions

Start the bundled fixture in terminal A:

```
cd repro/server
./run-http2-push.sh
```

In terminal B, verify that the PHP runtime has curl and HTTP/2 support, then run the control:

```
php repro/print_environment.php
php repro/push_callback_replacement.php control
```

Only after the control reaches `CALLBACK_INVOKED`, run the invalid case in a separate PHP process:

```
php repro/push_callback_replacement.php invalid
printf "PROCESS_EXIT=%s\n" "$?"
```

If the server is stopped, the request is refused, or the HTTP response code remains 0 without a push callback, the run is inconclusive. A missing push event cannot be interpreted as evidence that the crash is fixed.

The included transcript files preserve the observations as reported, with provenance notes. This package build did not have PHP 8.4.14 plus the curl extension and nghttpd available for independent execution.

2. Source-level hypothesis

The current upstream `ext/curl/multi.c` source provides a plausible explanation for the order-sensitive failure.

1

Destroy old callable cache

If `mh->handlers.server_push` is initialized, the setter calls `zend_fcc_dtor()`. That releases the PHP callable receiver and resets the callable-cache state.

2

Validate replacement

The setter validates the value supplied for `CURLMOPT_PUSHFUNCTION`. For `null`, validation fails and a `TypeError` is raised.

3

Return before native update

The invalid-callback branch returns before the later `curl_multi_setopt(..., _php_server_push_callback)` call and associated push-data setup.

4

Native callback path remains relevant

The current trampoline implementation later calls `zend_call_known_fcc()` on the stored PHP callable cache when Server Push is handled.

Source locations

Setter: lines around `CURLMOPT_PUSHFUNCTION` in current master. Callback trampoline: `_php_server_push_callback()`.

php-src/ext/curl/multi.c: callback and setter

Important limitation: current master source is not proof of the exact code in the reported PHP 8.4.14 binary. Capture the build's source commit or package metadata, compare the corresponding branch, and re-run on current supported patch releases.

The source order gives a falsifiable hypothesis: a rejected replacement could leave the C callback registered while the PHP-side callable cache is no longer initialized. The segmentation fault alone cannot tell us whether the failure is an invalid callable invocation, use-after-free, uninitialized state, or another defect. An ASan trace or equivalent core analysis is needed to classify the fault.

3. Security impact and lifecycle context

The phrase "lifecycle bug" can describe many very different conditions. The relevant invariant is that a callback pointer registered with a native library and the corresponding language-runtime callback state must remain mutually consistent for the whole period in which the native callback can run.

Question	Current answer
Can it crash a PHP process?	Yes, a SIGSEGV / exit 139 was reported in one local scripted sequence.
Is use-after-free proven?	No. The receiver destructor log indicates the PHP object was released, but that alone does not prove the later native call dereferenced freed memory.
Is remote reachability proven?	No. The sequence requires application code to try an invalid callback replacement and continue using that same multi handle. The remote peer's ability to reach that precondition is unknown.
Is RCE or critical severity proven?	No. There is no memory-safety trace, primitive, attacker-reachability evidence, or impact demonstration beyond process crash.
Is current PHP still affected?	Unknown. The observation is from PHP 8.4.14; the report has not been run against current supported patch releases.

Why cite the October 2026 PHP vulnerabilities?

The PHP disclosures released on 21 October 2026 describe critical vulnerabilities involving OOM cleanup and stale worker-global state; the 23 October advisory covers a separate, configuration-dependent session upload-progress scenario. They demonstrate why researchers inspect interrupted initialization, cleanup symmetry, and state consumers before and after a fix.

The technical paths differ from the cURL callback candidate. The connection is methodological only: in each audit, the researcher identifies what owns a piece of state, when it becomes valid, what happens on an exceptional path, and whether later code can still observe it. Severity must be independently established for each finding.

CVE-2026-109618: RCE via OOM corruption

CVE-2026-109619: RCE via stale global state

CVE-2026-115560: session upload-progress advisory

Do not transfer the Critical rating of the October CVEs to this callback crash. A similar lifetime concept does not establish equivalent exploitability or impact.

4. Prior art, remediation direction, and next milestones

The repository deliberately records earlier related cURL push-callback work. PHP has previously addressed a crash associated with an invalid callback value, and issue history documents other instability involving `CURLMOPT_PUSHFUNCTION`. These references make a novelty check essential; they do not by themselves resolve whether the exact sequence shown here is covered by a prior fix.

php-src commit history search for CURLMOPT_PUSHFUNCTION

php-src issue #16723: push-function issues on Ubuntu 24.04 / libcurl 8.5.0

Conservative hardening hypothesis

Validate a replacement callback in a temporary callable cache first. Only after validation succeeds should the implementation release the previous cache and commit the new one. The implementation should define how the PHP-side cache and native libcurl callback registration behave when validation or native registration fails. If invalid input is intended to clear the callback, unregister or neutralize the native callback safely rather than leaving an incoherent half-state.

This is a design direction, not a tested patch. Any proposed implementation must account for callback references, repeated replacement, failed native registration, cleanup ordering, and API compatibility.

Validation checklist

- Re-run the control and invalid case with nghttpd active; capture server logs, PHP runtime details, and exact exit codes.
- Record the PHP package version, source commit/build ID, compiler flags, curl_version() output, and nghttpd version.
- Re-run on a current supported PHP patch release; the October 2026 release archive lists PHP 8.4.27.
- Use ASan or another instrumented build to distinguish invalid callback state from use-after-free or other memory corruption.
- Add a php-src PHPT regression test for valid callback -> invalid replacement -> push, and keep a successful control.
- Review prior fixes and existing tests; submit privately through the current PHP security channel before making a CVE claim.

Current status

The useful contribution today is a compact reproducer, the matching successful control, the observed crash transcript, and a falsifiable source-level hypothesis. The next milestone is independent reproduction on supported builds plus a memory-safety trace and reachability analysis.

References

PHP source: ext/curl/multi.c, callback and setter path

PHP advisory, CVE-2026-109618

PHP advisory, CVE-2026-109619

PHP informational advisory, CVE-2026-115560

PHP 2026 news archive / security release information

php-src issue #16723

Prepared for technical review. No confirmed vulnerability identifier or severity is claimed.